1. COS226, Midterm f25, 5

Các phát biểu sau đúng hay sai?

A. Mergesort thực hiện ~2n log n phép so sánh khi sắp xếp mảng bất kỳ gồm 2n phần tử (cả trường hợp tốt nhất và xấu nhất)
B. Để tìm một key trong một mảng đã được sắp xếp độ dài 3n, một thuật toán sử dụng so sánh phải thực hiện $\Omega(\log n)$ phép so sánh trong trường hợp tồi nhất
C. Xét bài toán sắp xếp một mảng n phần tử mà các khóa chỉ gồm 4 giá trị. Có thể thiết kế một thuật toán sử dụng không quá 4n phép so sánh trong trường hợp xấu nhất
D. Xét một heap max với ít nhất 8 key. Gọi x là một key nằm tại một nút ở độ sâu 2 (nút gốc ở độ sâu 0) và y là key của một nút ở độ sâu 4. Khi đó, chắc chắn x >=y
E. Cho hai cây tìm kiếm nhị phân, mỗi cây chứa n key. Giả sử 2n key này là phân biệt. Có thể tạo một cây tìm kiếm nhị phân chứa 2n key đó bằng không quá 2n phép so sánh.
F. Giả sử cần thêm một key vào một cây BST đỏ đen nghiêng trái. Trong quá trình chèn, có thể có chuỗi liên tiếp hai (hoặc hơn) bước xoay trái (không có lật mầu hay xoay phải xen vào giữa)

2. COS226, Midterm s25, 5

Các phát biểu sau đúng hay sai?

A. Cho hai danh sách liên kết đơn, mỗi danh sách chứa n phần tử theo thứ tự đã sắp xếp, có thể tạo một danh sách liên kết đơn chứa 2n phần tử nói trên theo thứ tự đã sắp xếp với thời gian chạy cỡ $\Theta(n)$ và sử dụng bộ nhớ bổ sung cỡ $\Theta(1)$
B. Xét một mảng thay đổi kích thước được (resizable array) chứa n giá trị loại double. Loại mảng này tăng kích thước gấp bốn lần khi đầy, và giảm một nửa kích thước khi chỉ chứa ⅓ dung lượng. Với loại mảng này, kích thước bộ nhớ trong trường hợp xấu nhất là ~24n
C. Có thể tráo ngẫu nhiên một mảng trong thời gian $\Theta(n)$ ở trường hợp xấu nhất bằng cách đẩy (enqueue) n phần tử vào một RandomizedQueue (loại hàng đợi mà phép dequeue lấy ra phần tử ngẫu nhiên trong hàng đợi) sau đó lấy n phần tử ra bằng hàm dequeue.
D. Cho cây BST đỏ đen với n key phân biệt, có thể tạo một heap chứa n key đó với O(n) phép so sánh.
E. Giả sử cần thêm một key vào một cây BST đỏ đen nghiêng trái. Trong quá trình chèn, mỗi phép xoay phải đều có một phép đổi màu thực hiện ngay sau đó.

3. COS226, Midterm f25, 6

Thiết kế một thuật toán tìm index của giá trị 1 đầu tiên trong một mảng đã sắp xếp chứa toàn 0 và 1. Để đơn giản, giải thiết rằng mảng chứa ít nhất một giá trị 0 và một giá trị 1. Ví dụ trong mảng dưới đây, chỉ số của giá trị 1 đầu tiên là 3.

0	0	0	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7	8	9

Hãy hoàn thiện cài đặt sau của thuật toán trên

```
// precondition: a[] is sorted and contains only 0s and 1s,
//               with at least one 0 and at least one 1
public static int indexOfFirstOne(int[] a) {
    int lo = 0;
    int hi = a.length - 1;
    // invariants: a[lo] = 0 and a[hi] = 1
    while ( 1 ) {
        int mid = lo + (hi - lo) / 2;
        if (a[ 2 ] == 3 ) lo = 4 ;
        else              hi = 5 ;
    }
    return 6 ;
}
```

A. 0
B. 1
C. hi >= lo
D. hi > lo
E. hi > lo + 1
F. lo
G. lo + 1
H. mid - 1
I. mid
J. mid + 1
K. hi - 1
L. hi

4. COS226, Midterm f25, 7

Thiết kế một cấu trúc dữ liệu UniStack chứa dữ liệu kiểu String và cung cấp hai phương thức push và pop truyền thống của ngăn xếp nhưng trường hợp các phần tử trùng nhau được xử lý như sau: nếu một string đang được push vào stack trùng giá trị với một string đã có sẵn trong stack thì string cũ bị lấy ra khỏi stack. Ví dụ sử dụng:

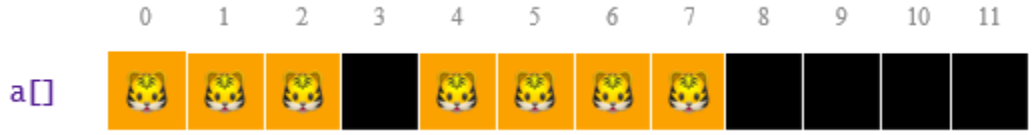
```
UniStack stack = new UniStack();      //  [ ]
stack.push("A");                      //  [ A ]
stack.push("B");                      //  [ B A ]
stack.push("C");                      //  [ C B A ]
stack.push("D");                      //  [ D C B A ]
stack.push("B");                      //  [ B D C A ]
stack.size();                         //  [ B D C A ]  returns  4 (not 5)
stack.pop();                          //  [ D C A ]    returns  B
stack.pop();                          //  [ C A ]      returns  D
stack.pop();                          //  [ A ]        returns  C
stack.pop();                          //  [ ]          returns  A (not B)
```

Yêu cầu hiệu năng:
Bộ nhớ sử dụng phải là O(n), với n là số phần tử trong UniStack
Mỗi thao tác push/pop tốn thời gian O(log n) trong trường hợp tồi nhất

5. COS226, Midterm s25, 7

Xét một mảng a[] độ dài n với các phần tử tô mầu đen hoặc cam, a[0] mầu cam và a[n-1] đen. Hãy thiết kế một thuật toán tìm chỉ số i sao cho a[i] mầu cam và a[i+1] mầu đen. Thuật toán phải có độ phức tạp O(log n) trong trường hợp tồi nhất.

Ví dụ: Nếu nếu mảng a[] chứa 12 phần tử sau đây thì thuật toán phải trả về i = 2 hoặc i = 7



6. COS226, Midterm f24, 8

Mảng dạng núi là một mảng có độ dài 2n, trong đó nửa đầu (n phần tử đầu tiên) được sắp xếp theo thứ tự tăng dần và nửa sau (n phần tử cuối cùng) được sắp xếp theo thứ tự giảm dần. Dưới đây là ví dụ về một mảng số nguyên dạng núi với n = 6.

4 7 8 11 18 20 | 31 7 7 4 2 1

(a) Hãy thiết kế một thuật toán sắp xếp mảng dạng núi chứa các số nguyên. Thuật toán phải có thời gian chạy O(n) trong trường hợp tồi nhất.

(b) Xét một phương thức có khai báo như sau: phương thức này nhận đầu vào là một mảng gồm 2n phần tử thuộc kiểu int và sắp xếp lại mảng đó thành một mảng có dạng "hình ngọn núi". Lưu ý rằng có thể tạo ra nhiều cấu hình dạng "hình ngọn núi" từ cùng một mảng ban đầu, và phương thức có thể trả về bất kỳ cách sắp xếp nào trong số các khả năng đó.

```
void makeMountainLike(Comparable[] a)
```

Hỏi có thể cài hàm đó với thời gian chạy O(n) được không?

7. COS226, Midterm f24, 9

Thiết kế một cấu trúc dữ liệu tên Tournament cho phép lưu trữ và tra cứu thông tin của các đấu thủ trong một giải đấu. Lớp Tournament cần có ba phương thức sau:

- insert() thêm dữ liệu của một cầu thủ
- getScore() trả về điểm của một cầu thủ, và
- getLead() trả về cầu thủ có điểm cao nhất

public class Tournament

public Tournament()	*creates an empty collection*
void insert(String name, int score)	*adds a player to the collection given their name and score*
int getScore(String name)	*returns the player's score given their name*
String getLead()	*returns the name of the player with the highest score*

Để đơn giản, bạn có thể dùng giả thiết rằng không có hai cầu thủ nào trùng tên.
Ví dụ. Sau đây là một chuỗi các thao tác

```
Tournament t = new Tournament();  //       [ ]
t.insert("Roger", 219);           // [ ("Roger", 219) ]
t.insert("John", 38);             // [ ("Roger", 219), ("John", 38) ]
t.getScore("John");               // returns 38
t.getLead();                      // returns "Roger"
t.insert("Freddie", 467);         // [ ("Roger", 219), ("John", 38), ("Freddie", 467) ]
t.getLead();                      // returns "Freddie"
t.insert("Brian",  189);          // [ ("Roger", 219), ("John", 38),
                                  //       ("Freddie", 467), ("Brian", 189) ]
t.getScore("Roger");              // returns 219
t.getLead();                      // returns "Freddie"
```

Lưu ý: các cặp nằm trong dấu ngoặc vuông biểu thị các đấu thủ hiện có trong tập hợp, nhưng API không yêu cầu bạn phải lưu trữ chúng theo bất kỳ thứ tự cụ thể nào.
Yêu cầu:
Constructor (hàm tạo) phải chạy trong thời gian hằng số
các phương thức insert(), getScore(), và getLeader() chạy thời gian O(log n) trong trường hợp tồi nhất
kiểu dữ liệu dùng bộ nhớ phụ kích thước O(n)